PROCEDIMIENTO Y DOCUMENTACIÓN CONSOLIDADA DE PRUEBAS

RENTAMOVIL

Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana Ramírez Vargas

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

1. CONTROL DE CAMBIOS

Versión	Fecha	Descripción	Responsable
1.0	2026-10-16	Consolidación del procedimiento, inventario y resultados de pruebas de los seis módulos backend Java, incluido `rtm-telemetry-gps` y la estandarización de las pruebas a inglés.	Equipo de desarrollo

2. OBJETIVO

Establecer un procedimiento común, repetible y verificable para diseñar, implementar, ejecutar y registrar las pruebas de los servicios backend de RentaMóvil. El documento consolida el inventario actual, los resultados por módulo, las limitaciones observadas y las acciones recomendadas.

Se documentan capacidades existentes y verificadas. Las pruebas sugeridas se presentan como mejoras pendientes, no como cobertura ya implementada.

3. ALCANCE

Este documento cubre los seis módulos backend Java:

- `rtm-iam`: identidad y acceso.
- `rtm-fleet-maintenance`: flota y mantenimiento.
- `rtm-booking-reservation`: reservas y rentas.
- `rtm-payment-billing`: pagos y facturación.
- `rtm-api-gateway`: entrada y enrutamiento de solicitudes.
- `rtm-telemetry-gps`: telemetría GPS y seguimiento de rentas en curso.

Incluye pruebas unitarias de dominio y casos de uso, pruebas directas de componentes web/filtros, dos suites de repositorios JPA con H2 (Fleet y Telemetry GPS) y una referencia separada a la verificación manual de integración con Postman.

No certifica todos los endpoints HTTP, la compatibilidad completa con SQL Server, los contratos entre servicios, el rendimiento, la seguridad integral ni flujos E2E automatizados. Las aplicaciones web y móvil y los repositorios de migraciones de base de datos no forman parte de esta línea base automatizada.

4. COMPONENTES Y ENTORNO DE PRUEBA

Módulo	Java	Spring Boot	Persistencia de producción
`rtm-iam`	21	4.1.1	JPA / SQL Server
`rtm-fleet-maintenance`	21	4.1.1	JPA / SQL Server
`rtm-booking-reservation`	21	4.1.1	JPA / SQL Server

Módulo	Java	Spring Boot	Persistencia de producción
`rtm-payment-billing`	21	4.1.1	JPA / SQL Server
`rtm-api-gateway`	21	4.0.8	No administra persistencia de negocio
`rtm-telemetry-gps`	21	4.1.1	JPA / SQL Server

Las versiones principales de herramientas verificadas son Java 21 y Maven 3.9.16. Los módulos usan JUnit Jupiter mediante `spring-boot-starter-test`; Mockito y AssertJ aparecen en las pruebas existentes. Fleet y Telemetry GPS añaden además H2 y el módulo de pruebas JPA de Spring Boot 4 en scope `test` (`spring-boot-starter-data-jpa-test` en Fleet, `spring-boot-data-jpa-test` en Telemetry GPS; ambos resuelven a la misma funcionalidad — en Spring Boot 4.x `@DataJpaTest` se movió a un módulo separado y ya no viene incluido en `spring-boot-starter-test`).

Ejecutar con JDK 21, que es la versión declarada en los POM. En este entorno, una ejecución con el JDK 27 instalado como predeterminado del sistema falló durante la inicialización del procesador de anotaciones de Lombok (`ExceptionInInitializerError` sobre `EndPosTable`); la línea base se obtuvo fijando `JAVA_HOME` al JDK 21 explícitamente en cada ejecución, sin cambiar el JDK por defecto del sistema operativo.

5. ESTRATEGIA DE PRUEBAS

5.1 Dominio y casos de uso

Probar reglas de negocio con JUnit 5, aislar dependencias externas con dobles cuando corresponda y comprobar resultados, errores e interacciones relevantes. No levantar Spring para pruebas unitarias de lógica que pueda probarse directamente.

Las pruebas de los seis módulos siguen un estándar de idioma común: nombres de métodos, anotaciones `@DisplayName` y comentarios en inglés, independientemente de que el código de producción de un módulo esté en español (es el caso de `rtm-iam`, `rtm-booking-reservation`, `rtm-payment-billing`, `rtm-api-gateway` y `rtm-telemetry-gps`; solo `rtm-fleet-maintenance` tiene también su código de producción en inglés). La única excepción deliberada son las aserciones que verifican literalmente el texto de un mensaje de excepción o notificación generado por el código de producción (por ejemplo `.hasMessageContaining("posterior")` en `rtm-booking-reservation`): ese fragmento se deja en español porque es exactamente lo que el código real produce, y traducirlo invalidaría la prueba. El código de producción en sí no fue traducido; es una decisión separada, pendiente.

5.2 Controladores y filtros

Algunas pruebas invocan métodos de controladores directamente o prueban filtros como unidades aisladas. Sirven para validar delegación y lógica local, pero no atraviesan el servidor MVC ni verifican por sí solas rutas HTTP, serialización JSON, validación de entrada o manejo HTTP integrado.

No se encontraron pruebas `@WebMvcTest`/`MockMvc` en los seis módulos. Si se necesita validar contratos HTTP, añadir pruebas MVC enfocadas.

5.3 Persistencia

Fleet y Telemetry GPS contienen cada uno una suite `@DataJpaTest` con H2. Fleet cubre consultas de sucursales, vehículos, mantenimientos y modelos. Telemetry GPS cubre el catálogo de dispositivos GPS y las dos consultas `@Query` de ubicación (`findLatest`, `findRoute`) que acotan la búsqueda a la ventana de cada renta — el mecanismo central de `Location.INV-001`/`INV-002`. H2 valida comportamiento JPA y JPQL, pero no sustituye SQL Server ni cubre todas sus particularidades.

IAM, Booking y Billing usan JPA en producción, pero no se encontraron pruebas `@DataJpaTest`, H2 o Testcontainers en sus suites actuales. Sus repositorios, mappings y consultas no se consideran verificados automáticamente por esta línea base.

La suite JPA de Fleet pasa, aunque Hibernate registra un aviso al intentar crear `shared.audit` porque el esquema compartido no está presente en H2; la auditoría y ese esquema quedan fuera de la cobertura de la suite. La suite de Telemetry GPS evita ese aviso porque sus pruebas no fijan un esquema de base de datos (`hibernate.default_schema` vacío en `src/test/resources/application.properties`), a costa de no validar el esquema `telemetry` real de producción.

5.4 Integración del sistema

La guía de Fleet documenta una verificación manual con Docker, SQL Server, gateway y Postman: 37 de 37 solicitudes exitosas el 4 de octubre de 2026. Es evidencia manual fechada; no forma parte de `mvn test` ni se ha repetido en las ejecuciones automatizadas posteriores. No existe una verificación manual equivalente documentada para `rtm-telemetry-gps` a la fecha de esta línea base.

6. PROCEDIMIENTO

6.1 Analizar el cambio

1. Inspeccionar el POM, la estructura de paquetes, las clases fuente y las pruebas cercanas.
2. Identificar el comportamiento existente, los contratos, las dependencias y las excepciones reales.
3. Seleccionar el nivel de prueba que corresponda; no inventar entidades, endpoints ni respuestas.
4. Determinar los límites de lo que la prueba puede demostrar, en especial si utiliza mocks o una base embebida.

6.2 Implementar pruebas

1. Colocar las pruebas bajo `src/test/java`, respetando el paquete del código probado.
2. Aislar casos de uso de Spring y utilizar mocks para dependencias externas cuando sea apropiado.
3. Probar componentes web directamente solo para su lógica local; usar `MockMvc` si se requieren garantías HTTP.

4. Para repositorios JPA, utilizar un slice de persistencia con datos mínimos y documentar la diferencia frente a SQL Server.
5. Cubrir caminos exitosos, alternos y errores relevantes al cambio, con aserciones sobre valores e interacciones.
6. Escribir nombres de métodos, `@DisplayName` y comentarios en inglés, salvo las aserciones que verifiquen literalmente un mensaje real del código de producción (ver 5.1).

6.3 Ejecutar desde la raíz de rentacar

Los repositorios son proyectos Maven independientes; no hay un POM agregador común. Ejecutar cada módulo por separado:

```
mvn -f rtm-iam/pom.xml test
mvn -f rtm-fleet-maintenance/pom.xml test
mvn -f rtm-booking-reservation/pom.xml test
mvn -f rtm-payment-billing/pom.xml test
mvn -f rtm-api-gateway/pom.xml test
mvn -f rtm-telemetry-gps/pom.xml test
```

Si el JDK por defecto del sistema no es la versión 21 declarada en los POM, fijar `JAVA_HOME` explícitamente antes de cada comando, por ejemplo:

```
JAVA_HOME=/usr/lib/jvm/java-21-openjdk mvn -f rtm-telemetry-gps/pom.xml test
```

Para ejecutar una suite específica, añadir `-Dtest=NombreDeLaPrueba` antes de `test`. Los reportes de Maven Surefire quedan en `target/surefire-reports` dentro de cada módulo.

6.4 Evaluar y registrar

1. Confirmar compilación y `BUILD SUCCESS`.
2. Registrar cantidad de pruebas, fallos, errores y omitidas por módulo.
3. Investigar warnings de inicialización aunque no cambien el resultado de la suite.
4. Registrar fecha, Java, comando, rama/commit y alcance de la prueba.
5. Separar evidencia automatizada de pruebas manuales y describir sus límites.

7. INVENTARIO Y RESULTADOS POR MÓDULO

Módulo	Inventario y comportamiento ejercitado	Resultado verificado el 2026-10-16
`rtm-iam`	Once clases de prueba, centradas en casos de uso: registro, login/logout, tokens, perfil, correo, contraseña, roles y estado.	39 pruebas; 0 fallos; 0 errores; `BUILD SUCCESS`
`rtm-fleet-maintenance`	Quince clases de casos de uso, seis controladores por invocación directa, verificación JWT y una suite `@DataJpaTest` con H2 para sucursales, vehículos, mantenimientos y modelos.	116 pruebas; 0 fallos; 0 errores; 0 omitidas; `BUILD SUCCESS`
`rtm-booking-reservation`	Dominio de reservas/rentas/seguros, eventos de pago, expiración y controladores invocados directamente. Dos repositorios en memoria son auxiliares de prueba, no suites.	53 pruebas; 0 fallos; 0 errores; `BUILD SUCCESS`
`rtm-payment-billing`	Dos clases de prueba: entidad `Payment` y caso de uso `PaymentUseCaseImpl`.	14 pruebas; 0 fallos; 0 errores; `BUILD SUCCESS`
`rtm-api-gateway`	Tres clases de prueba: filtro JWT, filtro de correlation ID y limitador en memoria.	10 pruebas; 0 fallos; 0 errores; `BUILD SUCCESS`
`rtm-telemetry-gps`	Tres casos de uso (catálogo GPS, ubicación/tracking, reacciones a eventos de renta), cinco controladores por invocación directa, verificación JWT RS256, validador de API key de dispositivo, tracking en memoria, y persistencia con `@DataJpaTest`/H2 más adaptadores probados con Mockito.	57 pruebas; 0 fallos; 0 errores; `BUILD SUCCESS`
Total backend Java	**Ejecuciones independientes por módulo**	**289 pruebas; 0 fallos; 0 errores**

La suma de 289 es un agregado de seis ejecuciones Maven independientes, no una ejecución de reactor multi-módulo.

8. CRITERIOS DE ACEPTACIÓN

- El módulo afectado compila con Java 21 y su suite completa termina sin fallos ni errores.
- Los nombres y escenarios corresponden a clases y comportamiento existentes.
- Las pruebas son repetibles y no dependen de servicios remotos o credenciales reales, salvo en procedimientos manuales controlados.
- Una prueba de slice declara el motor de base de datos y las diferencias respecto a producción.
- Las comprobaciones manuales no se presentan como automatizadas.
- No se informa porcentaje de cobertura si no existe una medición configurada.

- Las pruebas nuevas y las existentes están en inglés (nombres de métodos, `@DisplayName`, comentarios), salvo las aserciones que verifiquen literalmente un mensaje real del código de producción en español.

9. BRECHAS Y RIESGOS CONOCIDOS

Prioridad	Brecha	Alcance afectado
Alta	Falta validación HTTP real con `@WebMvcTest`/`MockMvc`; pruebas directas no prueban transporte ni serialización.	Los seis módulos; especialmente controllers de Fleet, Booking y Telemetry GPS
Alta	No hay pruebas de repositorio en tres módulos JPA; Fleet y Telemetry GPS usan H2, no SQL Server, y no prueban `shared.audit` ni el esquema `telemetry` real.	IAM, Booking, Billing y límites de Fleet y Telemetry GPS
Alta	Billing tiene dos clases de prueba y 14 casos, el inventario más pequeño por servicio.	Billing
Media	No hay pruebas automáticas de rutas/upstreams del Gateway en contexto HTTP.	API Gateway
Media	JaCoCo no está configurado, así que no existe porcentaje de cobertura reproducible.	Todos
Media	No hay pipeline CI configurado para ejecutar las suites al integrar cambios.	Todos
Media	La verificación integrada de Postman es manual, no repetible mediante `mvn test`, y no existe una equivalente para Telemetry GPS.	Sistema completo; Telemetry GPS en particular
Baja	El código de producción permanece en español en cinco de los seis módulos, mientras las pruebas ya están en inglés; la inconsistencia de idioma entre prueba y código probado puede confundir a quien no conozca la decisión.	IAM, Booking, Billing, API Gateway, Telemetry GPS

10. ACCIONES DE MEJORA RECOMENDADAS

1. Priorizar pruebas de reglas de negocio y errores de pago en Billing.
2. Añadir pruebas MVC con `MockMvc` para endpoints y manejo de errores HTTP.
3. Añadir pruebas de persistencia para IAM, Booking y Billing, replicando el patrón `@DataJpaTest` + H2 de Fleet y Telemetry GPS; evaluar Testcontainers con SQL Server para compatibilidad real.
4. Completar pruebas del enrutamiento y upstreams del Gateway.
5. Configurar JaCoCo con metas acordadas y agregar `mvn test` a CI por repositorio.
6. Automatizar gradualmente los flujos Postman/E2E más críticos; documentar una verificación manual de integración para Telemetry GPS equivalente a la de Fleet.

7. Decidir si el código de producción se traduce a inglés para eliminar la inconsistencia de idioma entre prueba y código probado, o si se mantiene el español como estándar de producción de forma permanente.

Estas acciones son propuestas; no forman parte de los resultados ejecutados aquí.

11. ROLES Y RESPONSABILIDADES

Rol	Responsabilidad
Desarrollador	Implementar pruebas del cambio, ejecutarlas en su módulo y registrar los resultados.
Revisor técnico	Confirmar que los escenarios corresponden al código real y que se declaran sus límites.
Responsable de calidad o líder	Aprobar criterios de aceptación y priorizar las brechas.
Responsable del entorno	Mantener los servicios y datos de prueba requeridos para validaciones manuales o integradas.